

# INTRODUCTION

## 1.1 Background of the Study

Financial cooperatives (FinCoop systems) are widely used to manage member savings, loans, and transactions. These systems are often challenged by inefficiencies in handling large volumes of financial data, especially when using manual or poorly optimized digital systems.

As cooperative membership grows, transaction processing becomes slower, data retrieval becomes inefficient, and risks such as duplicate transactions and delayed loan processing increase.

This project proposes a Cooperative Financial Transaction Management System (FinCoop) that applies advanced data structures and algorithms to improve performance, scalability, and reliability.

## 1.2 Problem Statement

Traditional cooperative financial systems face several challenges including:

- Slow transaction processing when user numbers increase
- Difficulty in retrieving transaction history efficiently
- Duplicate transactions due to weak validation systems
- Inefficient loan processing and prioritization
- Poor scalability for large datasets (10,000–1,000,000+ records)

The FinCoop system is designed to solve these issues using optimized data structures such as hash tables, queues, stacks, trees, and heaps to ensure fast, accurate, and scalable financial operations.

## 1.3 Objectives of the Study

### General Objective

To design a scalable cooperative financial transaction system using efficient data structures and algorithms.

### Specific Objectives

- To design a system for managing financial transactions efficiently
- To implement fast transaction search using hash tables

- To detect and prevent duplicate transactions
- To prioritize loan processing using heaps
- To support undo/rollback operations using stacks
- To enable structured reporting using trees

## Use Case Diagram

### 1.4 Scope of the Study

The system focuses on:

- Member account management
- Deposits and withdrawals
- Loan applications and approvals
- Transaction history tracking
- Loan prioritization

The system does NOT include real banking integration or actual financial deployment.

## ALGORITHM AND DATA STRUCTURE DESIGN

### 3.1 Introduction

This chapter presents the data structures and algorithms used in the FinCoop system. The system is designed for efficiency, scalability, and reliability when handling financial transactions.

### 3.2 System Constraints

The system must:

- Handle up to 1,000,000+ transactions
- Ensure fast lookup (real-time response)
- Minimize memory usage
- Support concurrent financial operations
- Prevent duplicate transactions

### **3.3 Data Structure Selection and Justification**

#### **1. Array**

- Used for basic storage of transactions
- Simple but inefficient at scale

#### **2. Linked List**

- Used for dynamic transaction history
- Efficient insertion and deletion

#### **3. Queue**

- Used for processing transactions in FIFO order
- Ensures fairness

#### **4. Stack**

- Used for rollback/undo operations
- Stores recent transactions

#### **5. Hash Table (Key Structure)**

- Used for fast lookup ( $O(1)$ )
- Detects duplicate transactions
- Maps Transaction ID  $\rightarrow$  Data

#### **6. Tree (BST)**

- Used for sorted transaction records
- Enables range queries (date, amount)

#### **7. Heap (Priority Queue)**

- Used for loan prioritization
- Highest priority loans processed first

### **3.4 Core Algorithms**

#### **Insert Transaction Algorithms**

Step 1: Check Hash Table for duplicate

Step 2: If not duplicate  $\rightarrow$  insert into Array

Step 3: Add to Linked List

Step 4: Store in Hash Table

Step 5: Add to Queue for processing

### Search Transaction Algorithm

Step 1: Input Transaction ID

Step 2: Check Hash Table

Step 3: If found → return transaction

Step 4: Else → return NOT FOUND

### Process Transaction Algorithm

Step 1: Remove from Queue

Step 2: Execute transaction

Step 3: Push to Stack for rollback

Step 4: Update Tree and Hash Table

### 3.5 Complexity Analysis

| Structure   | Insert      | Search      | Update      | Delete      |
|-------------|-------------|-------------|-------------|-------------|
| Array       | $O(1)$      | $O(n)$      | $O(n)$      | $O(n)$      |
| Linked List | $O(1)$      | $O(n)$      | $O(n)$      | $O(n)$      |
| Hash Table  | $O(1)$      | $O(1)$      | $O(1)$      | $O(1)$      |
| Tree        | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ |
| Heap        | $O(\log n)$ | $O(n)$      | $O(\log n)$ | $O(\log n)$ |

### 3.6 Performance at Scale

- 1,000 transactions → All structures perform well
- 10,000 transactions → Hash tables become necessary
- 1,000,000 transactions → Trees + Hash tables essential

## CHAPTER 23: SYSTEM DESIGN

### 23.1 System Overview

- The FinCoop system is a conceptual financial management system that processes cooperative transactions using optimized data structures to ensure speed, accuracy, and scalability.

### 23.2 System Architecture

- **Flow of System**

Users



Input Layer (Transactions / Loans)



Processing Engine



Data Structures Layer

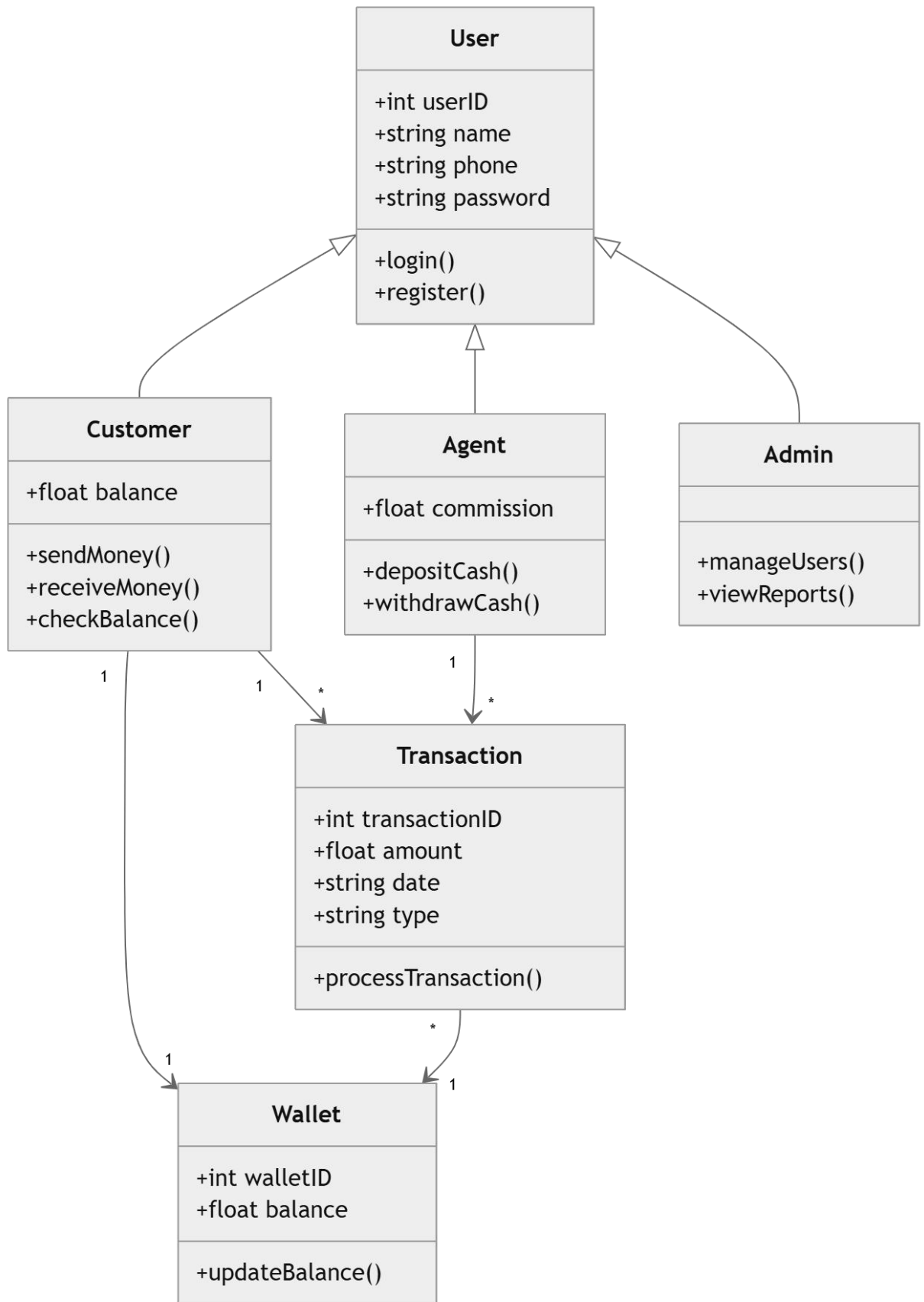
(Hash Table + Queue + Stack + Tree + Heap)



Output Layer



Reports / Account Updates / Loan Status



### **23.3 Input – Process – Output Model**

#### **Input**

- Loan requests
- Deposits
- Withdrawals

#### **Process**

- Validate transaction
- Check duplicates (Hash Table)
- Queue processing
- Prioritize loans (Heap)
- Store structured records (Tree)

#### **Output**

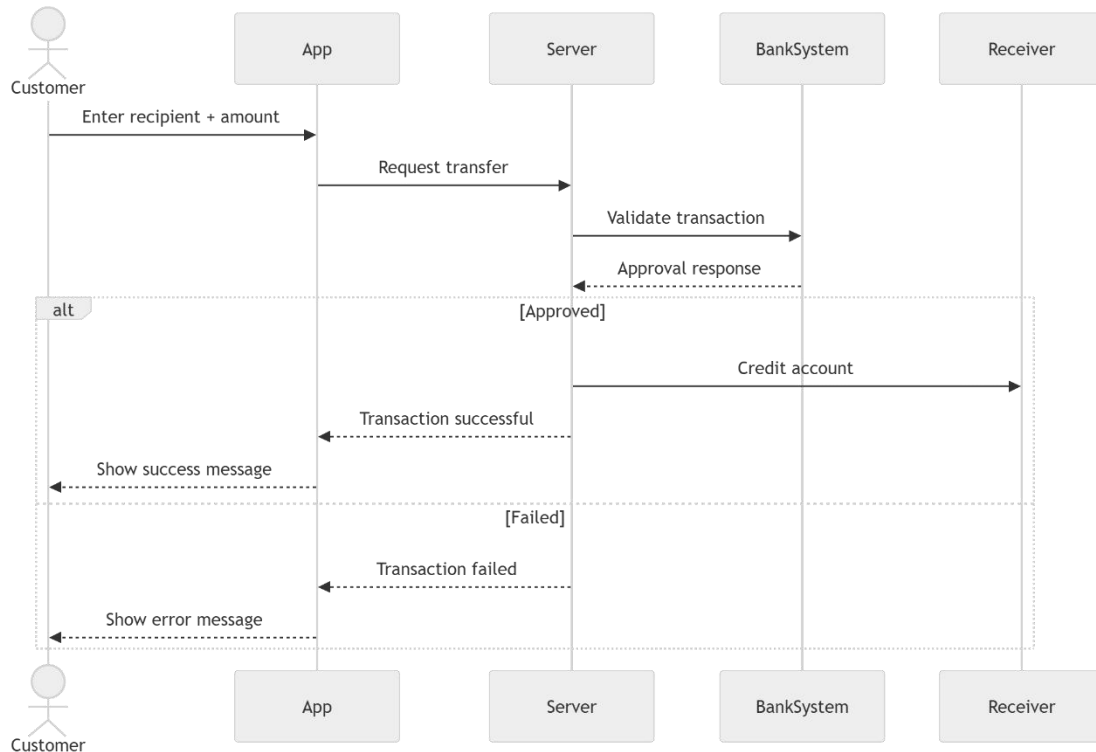
- Updated balances
- Loan status updates
- Transaction confirmations

### **23.4 System Bottlenecks**

- Array search becomes slow at scale
- Single queue may cause delays
- Tree balancing required for performance

### **23.5 Scalability Design**

- Hash tables scale for lookup efficiency
- Trees ensure structured data access
- Heaps optimize loan prioritization
- Distributed queues improve processing speed



## MERITS & LIMITATIONS

### Hash Table

#### Merit:

Extremely fast lookup

#### Demerit:

No sorting

### Tree

#### Merit:

Sorted data access

#### Demerit:

Slower than hash table

### Queue

#### Merit:

Fair processing

#### Demerit:

No random access

### Heap



**Merit:**

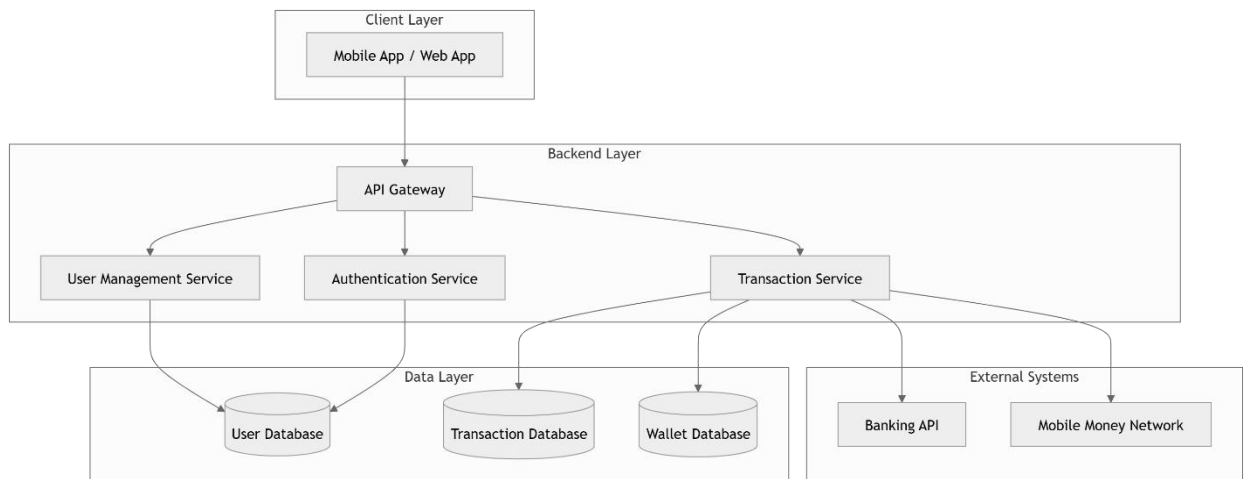
Priority handling

**Demerit:**

Complex structure

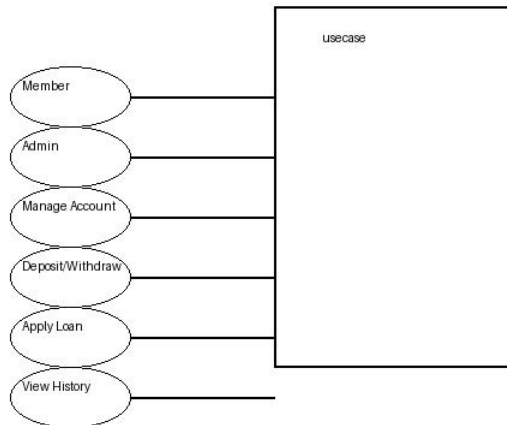
**COMPLEXITY INSIGHTS.**

- Best overall lookup: Hash Table ( $O(1)$ )
- Best sorted access: Tree ( $O(\log n)$ )
- Best prioritization: Heap ( $O(\log n)$ )
- Worst scaling structure: Array ( $O(n)$ )

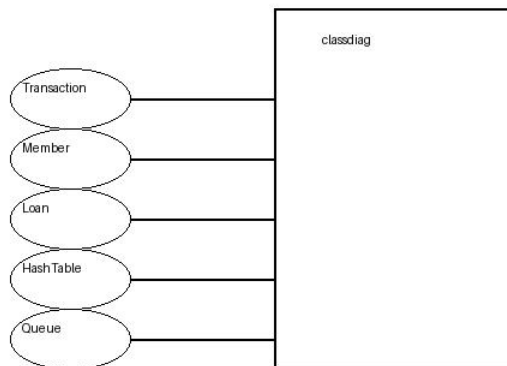
**Overall system design**

# UML DIAGRAMS

## Use Case Diagram



## Class Diagram



## Architecture Diagram

